SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	N.Adi Narayana	**Reg. No.:**	192525346
Section / Batch:	AIML	**Date:**	

Code of Conduct

I Cherukuri Likhitha(192424239) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Likhitha

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S  → NP VP
NP → Det N
VP → V NP

Det → the | a
N   → student | teacher | book
V   → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
        pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

CODE:

```python
# Experiment 1: Context-Free Grammar Rule Validation
# and Parse Tree Construction

def parse_cfg(sentence):
    # Split the sentence into words
    words = sentence.lower().split()

    # Grammar
    determiners = {"the", "a"}
    nouns = {"student", "teacher", "book"}
    verbs = {"reads", "likes"}

    # A valid sentence must contain exactly 6 words:
    # Det N V Det N
    if len(words) != 5:
        return "Invalid Sentence"

    # Extract words
    det1, noun1, verb, det2, noun2 = words

    # Check NP -> Det N
    if det1 not in determiners or noun1 not in nouns:
        return "Invalid Sentence"

    # Check VP -> V NP
    if verb not in verbs:
        return "Invalid Sentence"

    if det2 not in determiners or noun2 not in nouns:
        return "Invalid Sentence"

    # Construct parse tree
    parse_tree = {
        "S": [
            {
                "NP": [
                    {"Det": [det1]},
                    {"N": [noun1]}
                ]
            },
            {
                "VP": [
                    {"V": [verb]},
                    {
                        "NP": [
                            {"Det": [det2]},
                            {"N": [noun2]}
                        ]
                    }
                ]
            }
        ]
    }

    return parse_tree
```

```
    return parse_tree


# Function to display the parse tree
def print_tree(tree, level=0):
    for key, value in tree.items():
        print("  " * level + key)

        for child in value:
            if isinstance(child, dict):
                print_tree(child, level + 1)
            else:
                print("  " * (level + 1) + child)


# Main program
sentence = input("Enter a sentence: ")

result = parse_cfg(sentence)

if result == "Invalid Sentence":
    print("\nInvalid Sentence")
else:
    print("\nValid Sentence")
    print("\nParse Tree:")
    print_tree(result)
```

OUTPUT:

```
Enter a sentence: the student reads a book

Valid Sentence

Parse Tree:
S
  NP
    Det
      the
    N
      student
  VP
    V
      reads
    NP
      Det
        a
      N
        book
```

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he     → singular, third person
she    → singular, third person
it     → singular, third person
they   → plural

runs   → singular verb
writes → singular verb

run    → plural verb
write  → plural verb
```

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject	Verb	Expected
he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

CODE:

```python
# Experiment 2: Agreement and Feature Structure Checking

# Feature structures for subjects
subject_features = {
    "he": {
        "Number": "singular",
        "Person": "third"
    },
    "she": {
        "Number": "singular",
        "Person": "third"
    },
    "it": {
        "Number": "singular",
        "Person": "third"
    },
    "they": {
        "Number": "plural",
        "Person": "third"
    }
}

# Feature structures for verbs
verb_features = {
    "runs": {
        "Number": "singular"
    },
    "writes": {
        "Number": "singular"
    },
    "run": {
        "Number": "plural"
    },
    "write": {
        "Number": "plural"
    }
}


# Function to check subject-verb agreement
def check_agreement(subject, verb):

    # Convert input to lowercase
    subject = subject.lower()
    verb = verb.lower()
```

```
        "Number": "plural"
    }
}


# Function to check subject-verb agreement
def check_agreement(subject, verb):

    # Convert input to lowercase
    subject = subject.lower()
    verb = verb.lower()

    # Check whether subject and verb exist
    if subject not in subject_features:
        return False

    if verb not in verb_features:
        return False

    # Get feature structures
    subject_feature = subject_features[subject]
    verb_feature = verb_features[verb]

    # Compare Number feature
    if subject_feature["Number"] == verb_feature["Number"]:
        return True
    else:
        return False


# Main program
subject = input("Enter Subject: ")
verb = input("Enter Verb: ")

result = check_agreement(subject, verb)

print("\nSubject:", subject)
print("Verb:", verb)

print("Agreement:", result)
```

OUTPUT:

```
Enter Subject: he
Enter Verb: runs

Subject: he
Verb: runs
Agreement: True
```

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

```
S  → NP VP       [1.0]

NP → John        [0.6]
NP → Mary        [0.4]

VP → runs        [0.5]
VP → walks       [0.5]
```

Probability Formula

```
P(Sentence)
=
P(S→NP VP)
```

- × P(NP)
- × P(VP)

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

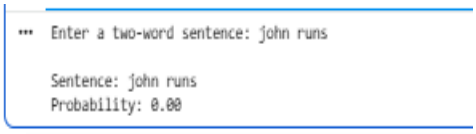
Test Cases

Input Sentence	Expected Probability
John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

CODE:

```
# Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

# PCFG Grammar
grammar = {
    "S": {
        ("NP", "VP"): 1.0
    },

    "NP": {
        ("John",): 0.6,
        ("Mary",): 0.4
    },

    "VP": {
        ("runs",): 0.5,
        ("walks",): 0.5
    }
}


# Function to calculate sentence probability
def sentence_probability(sentence):

    # Split sentence into words
    words = sentence.strip().split()

    # Sentence must contain exactly two words
    if len(words) != 2:
        return 0.0

    subject = words[0]
    verb = words[1]

    # Check NP probability
    if (subject,) not in grammar["NP"]:
        return 0.0

    # Check VP probability
    if (verb,) not in grammar["VP"]:
        return 0.0

    # Get probabilities
    s_probability = grammar["S"][("NP", "VP")]
    np_probability = grammar["NP"][(subject,)]
    vp_probability = grammar["VP"][(verb,)]

    # Calculate sentence probability
    probability = (
        s_probability *
        np_probability *
        vp_probability
    )

    return probability


# Main program
sentence = input("Enter a two-word sentence: ")

probability = sentence_probability(sentence)

print("\nSentence:", sentence)
print("Probability:", f"{probability:.2f}")
```

OUTPUT:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____